Software Requirements Specification for DAIIM: A Decentralized Marketplace for AI Inference

Team 6,
Robert Lam
Nathan Hum
Aidan Lao
Keira Laskoski

November 11, 2026

Contents

1 Introduction **1**
- 1.1 Purpose . . . 1
- 1.2 Scope . . . 1
- 1.3 Stakeholders . . . 2
- 1.4 Definitions and Acronyms . . . 3
- 1.5 Characteristics of Intended Reader . . . 4

2 System Overview **5**
- 2.1 Problem Statement . . . 5
- 2.2 Product Vision . . . 5
- 2.3 System Context . . . 6
  - 2.3.1 System Context Diagram . . . 6
  - 2.3.2 Responsibilities . . . 6
  - 2.3.3 Operating Context and Safety Significance . . . 7
- 2.4 User Characteristics . . . 8
- 2.5 System Constraints . . . 8

3 Stakeholder Requirements **8**

4 User Personas **10**
- 4.1 Persona 1: Emergency Management Coordinator (Task Requester) . . . 10
- 4.2 Persona 2: Independent Stock Forecaster (Inference Provider) . . . 10
- 4.3 Persona 3: Hydrology Researcher (Inference Provider) . . . 11

5 Use Cases **11**
- 5.1 UC1: Sign in to the web application. . . . 11
- 5.2 UC2: Sign out of the web application. . . . 12
- 5.3 UC3: Add funds to Web3 Wallet . . . 13
- 5.4 UC4: Browse marketplace information . . . 14
- 5.5 UC5: Create an inference task . . . 15
- 5.6 UC6: Register as an inference provider for a task . . . 15
- 5.7 UC7: Submit an inference request for a task . . . 17
- 5.8 UC8: Accept an inference request for a task . . . 19
- 5.9 UC9: Commit and reveal a prediction for an inference request . . . 20
- 5.10 UC10: Aggregate predictions and produce a final inference result . . . 21
- 5.11 UC11: Update and maintain provider reputation . . . 22
- 5.12 UC12: Distribute rewards to inference providers . . . 22
- 5.13 UC13: Cancel an inference request with too few participating providers . . . 23
- 5.14 UC14: Unregister as an inference provider for a task . . . 24
- 5.15 UC15: View transaction history and current balances . . . 25
- 5.16 UC16: Use in-app guidance to learn the system . . . 26

6 Functional Requirements **26**
6.1 Accounts and Authorization . . . 26
6.2 Inference Tasks . . . 27
6.3 Provider Registration and Staking . . . 27
6.4 Inference Requests . . . 28
6.5 Provider Acceptance and Predictions . . . 28
6.6 Aggregation and Results . . . 30
6.7 Reputation and Rewards . . . 30
6.8 Penalties . . . 31
6.9 Transparency . . . 32
6.10 Data Handling . . . 32
6.11 Safety Functional Requirements . . . 33

7 Data Requirements **33**
7.1 Major Data Entities . . . 33
7.2 Data Dictionary . . . 34
7.3 Privacy and Data Retention . . . 36

8 Non-Functional Requirements **37**
8.1 Accuracy . . . 37
8.2 Usability . . . 37
8.3 Security . . . 37
8.4 Reliability and Availability . . . 38
8.5 Maintainability . . . 38
8.6 Scalability . . . 38
8.7 Portability and Installability . . . 38
8.8 Safety . . . 39

9 AI and Machine Learning Requirements **39**
9.1 Evaluation Metrics . . . 39
9.2 Known Limitations . . . 39

10 Requirements Traceability Matrices **40**

11 Rationale **42**
11.1 On-Chain Execution . . . 42
11.2 Provider Staking . . . 42
11.3 Voluntary Unregistration and Stake Return . . . 43
11.4 Unresolved Reward Transfers . . . 43
11.5 Protocol-Wide Commit and Reveal Periods . . . 44
11.6 Commit-Reveal Privacy . . . 44
11.7 Request Broadcast and Collusion . . . 44

12 Likely Changes **45**

13 Unlikely Changes **45**

14 Appendices **47**
14.1 Appendix A: Additional Materials 47
14.1.1 Weighted-Median Aggregation Algorithm 47
14.1.2 Confidence-Based Reputation Update Algorithm 47
14.2 Appendix B: Development Plan 48
14.3 Appendix C: Values of Auxiliary Constants 48
14.4 Appendix D: Generative AI Disclosure 49
14.5 Appendix E: Reflection 50

Revision History

Date	Version	Notes
October 8	1.0	Aidan: Added sections 1.1 to 1.5 (Purpose, Scope, Stakeholders, Definitions and Acronyms, and Characteristics of Intended Reader)
October 11	1.0	Robert: Added major functional requirements
October 11	1.0	Aidan: Grouped functional requirements into categories and added functional requirements derived from the stakeholder requirements
October 11	1.0	Nathan: Added all of section 2, likely changes and unlikely changes
October 16	1.0	Robert: Added data and AI reqs sections
October 18	1.0	Keira: Added Sections 5, 8, 10
October 18	1.0	Keira: Revised sections 5, 8, 10 based on comments from team meeting
October 19	1.0	Robert: Fix typos, add acronyms, add subtitle, remove instructions
October 19	1.0	Keira: Added requirement traceability to use cases.
November 5	1.1	Robert: Added commit-reveal scheme - aligned related requirements

1 Introduction

1.1 Purpose

The current AI market is controlled by only a few mega-corporations, including NVIDIA, Anthropic, Microsoft, and OpenAI. These large companies host and control their own model, trained on massive datasets which can contain social. This causes an issue regarding AI centralization, which causes implications of a single point of failure, bias amplification, and a lack of trasnparency regarding the algorithms and data sets used.

DAIIM (Decentralized Artificial Intelligence Inference Marketplace) exists to reduce reliance on the small number of large organizations that currently provide most AI inference. It serves two groups of people.

For people who need machine learning predictions, DAIIM provides a way to obtain inferences for a task they define without trusting a single centralized provider. Each inference request is answered by multiple independent providers, and their predictions are combined into one consensus result. Because no single provider controls the outcome, the result is more reliable, harder to censor or manipulate, and not dependent on any one point of failure.

For people who build machine learning models, DAIIM provides an open marketplace where they can sell predictions from their models and earn money for accurate results. This gives independent providers a way to compete with large organizations and a reason to keep improving their models.

1.2 Scope

DAIIM allows task requesters to obtain reliable inferences by combining predictions from multiple independent inference providers.

The inputs to the system consist of task definitions, inference requests, and network participation. Task definitions are specifications of a problem to be solved via machine learning, including the expected input and output types. Inference requests ask the network to perform inference for a given task and include input data or some specification of where to find the input data. Network participation includes machine learning predictions for individual inference requests that are supplied to the network by an individual. The system outputs aggregated inference results and maintains information about the network. The aggregated inference results include the final prediction for a given inference request, as well as information about the quality of the prediction.

The expected benefits are reduced reliance on centralized AI providers, improved reliability of inferences through redundant predictions from independent providers, and incentives for providers to improve the quality, performance, availability, and cost-effectiveness of their models.

The following are outside the scope of DAIIM:

- Inference tasks whose output is not a single numeric value, such as predicting a category label. Support for additional model types is a stretch goal.

- Inference on private or sensitive data within Trusted Execution Environments (TEEs). This is a stretch goal.
- DAIIM doesn't train models. It just uses model providers' existing models to make predictions for inference requests.
- Using a custom blockchain. DAIIM will use Solana, an existing public blockchain.

1.3 Stakeholders

Primary Stakeholders

Primary stakeholders interact with DAIIM directly.

- **Task requesters**
  Task requesters are people who are requesting inferences to be made for a given task, using their own input data. These people value consensus and reliability: they want a prediction that combines the results of multiple independent models, rather than relying on a single model that may be wrong, biased, or unavailable. For example, emergency management agencies would make requests to predict river levels or wildfire risk, where a single incorrect forecast can leave a community unprepared. Similarly, decentralized finance platforms could make requests to predict stock prices, where relying on a single source of predictions creates a risk of error or manipulation. Because input data is shared with inference providers, task requesters are expected to submit data that is not sensitive.
- **Inference providers**
  Inference providers are people with computing resources and models, and want to monetize their regression models. These people usually have specialized or powerful models that can be used to make predictions for a given task.

Secondary Stakeholders

Secondary stakeholders do not use DAIIM directly, but are affected by the inferences it produces or by how it handles data.

- **Beneficiaries of machine learning inference**
  Beneficiaries are people and organizations, such as laboratories, that rely on the results of machine learning inference without requesting inferences themselves. For example, a laboratory may make decisions based on biomarker predictions that a healthcare researcher obtained through DAIIM. These people benefit from inferences that are reliable and not controlled by a single provider.
- **Regulatory bodies**
  Regulatory bodies are organizations that oversee machine learning inference and data

privacy. They want to ensure that personal information and sensitive data handled by DAIIM, such as inference inputs and training data, is only accessible to the parties that need it, is encrypted, and is not retained longer than necessary to fulfill an inference request.

1.4 Definitions and Acronyms

Term	Definition
AI	Artificial Intelligence.
ML	Machine Learning.
MAE	Mean Absolute Error, the arithmetic mean of the absolute differences between predicted and ground-truth values.
JSON	JavaScript Object Notation, a text-based data format used to represent structured data.
DAIIM	Decentralized Artificial Intelligence Inference Marketplace, the system specified in this document.
Task	A problem to be solved via machine learning, such as predicting a biomarker level or a stock price.
Task definition	A specification of a task, including the expected input and output types.
Inference	A prediction produced by a machine learning model for a given input.
Inference request	A request for the network to perform inference for a given task, including input data or a specification of where to find the input data.
Task requester	A person who requests inferences to be made for a given task.
Inference provider	A person with computing resources and models who supplies predictions for inference requests to the network.
Network participation	Machine learning predictions for individual inference requests supplied to the network by an inference provider.
Aggregated inference result	The final prediction for a given inference request produced by combining the predictions of multiple inference providers, together with information about the quality of the prediction.
Ground truth	The true, observed value that a prediction is attempting to estimate.
Linear regression	A machine learning model that predicts a numeric output as a linear combination of its input features.
Decentralized	Operating without a single central organization that coordinates or controls the system.

Blockchain	A distributed, append-only ledger of transactions that is maintained by a network of nodes rather than a central authority.
On-chain	Recorded on, or executed by, a blockchain.
Web3	Applications built on blockchains, where users interact with the system through their own blockchain accounts and authorize actions by signing transactions.
Commitment	A cryptographic mechanism that allows someone to commit to a value while keeping it hidden, with the ability to reveal and prove the original value later.
Reveal	The submission of a prediction and its salt for verification against a prior commitment.
Salt	A random value included in a commitment to make the hash unique and less prone to reversal.
Hash	A fixed-length output generated by a cryptographic hash function from input data, designed to be deterministic and infeasible to reverse.
Token	A digital asset on a blockchain, used by DAIIM to compensate inference providers.
TEE	Trusted Execution Environment. A secure area of a processor that protects the code and data loaded inside it from unauthorized access.
TLS	Transport Layer Security. A cryptographic network protocol for protecting data in transit and support data authentication and integrity.
PIPEDA	Personal Information Protection and Electronic Documents Act. A Canadian federal act defines how to protect personal information that is collected, used, or disclosed.

Table 1: Definitions and Acronyms

1.5 Characteristics of Intended Reader

The intended readers of this document are the software engineers, reviewers, and supervisors who will design, implement, verify, and maintain DAIIM. Readers are expected to have:

- An undergraduate-level background in software engineering, including requirements specification, software design, and testing, equivalent to the third year of a software engineering degree.
- Introductory knowledge of machine learning, including supervised learning, linear regression, model training versus inference, and basic error measures such as relative error, equivalent to an introductory undergraduate machine learning course.

- Introductory knowledge of blockchain and Web3 concepts, including blockchain transactions, accounts and wallets, transaction signing, and tokens. Familiarity with smart contract development is helpful but not required.
- Familiarity with web applications that run in a modern browser.

Terms specific to the domain of DAIIM are defined in Table 1.

2 System Overview

This section provides general information about the system. It identifies the problem, the vision, and the interfaces between the system and its environment. This section also describes user characteristics and lists the system constraints.

2.1 Problem Statement

AI inference services often rely on centralized providers, which can create single points of failure and leave users dependent on a single service for their results. Users may also have limited visibility into how results are obtained, how reliable they are, and whether the service provider is operating as expected. These limitations raise concerns about service availability, trust, and the ability to assess the quality of inference results. Relying on a single provider also limits opportunities for independent providers to contribute their inference capabilities to a shared marketplace. DAIIM addresses the need for an approach to AI inference that supports greater reliability, accountability, and participation. The underlying problem is how to provide users with confidence in inference results while allowing multiple providers to participate and be evaluated fairly, without requiring users to rely entirely on one centralized service.

2.2 Product Vision

DAIIM aims to make AI inference more reliable, transparent, and accessible by connecting users who need predictions with independent providers who can supply them. By combining predictions from multiple providers and making relevant performance and transaction information available, the marketplace seeks to build trust in inference results while creating opportunities for providers to offer their services. The system's goals are to:

- Improve the reliability of inference results by combining predictions from multiple independent providers, assessing the quality of their contributions, and reducing dependence on any single provider.
- Create a fair and accountable marketplace in which providers can build reputations and receive compensation based on their performance.

- Make marketplace activities transparent by allowing users to inspect aggregated results, relevant provider performance information, and records of transactions and payments.
- Make AI inference accessible to users without requiring prior knowledge of AI or Web3 technologies, while giving inference providers a way to offer their services and earn compensation for their contributions.

2.3 System Context

2.3.1 System Context Diagram

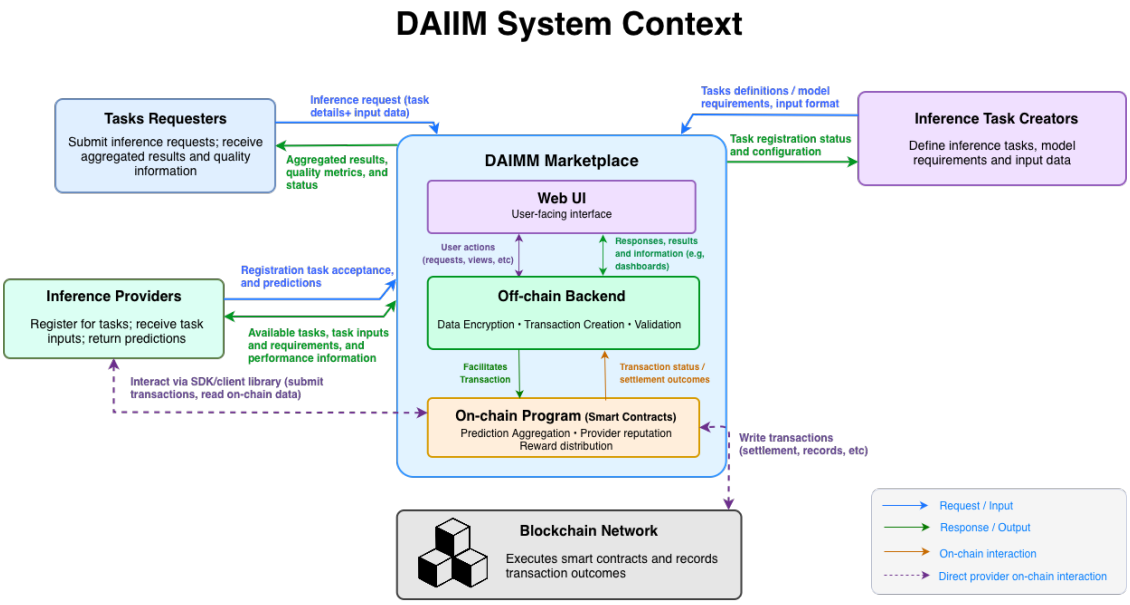


Figure 1: System Context

2.3.2 Responsibilities

- **Task Requester Responsibilities:**
  - Define the inference task by specifying the problem to be solved, the expected input and output formats, and other required task settings.
  - Submit requests for predictions using an existing task definition and input data that conforms to the task's requirements.
  - Specify the amount of tokens offered for an inference request and authorize the associated payment transaction.
  - Receive the final aggregated prediction, information about the range of provider predictions, the number of providers that submitted predictions, and the request status.

- View the status and results of previously submitted inference requests.

- **Inference Provider Responsibilities:**
  - Register to provide predictions for an inference task by staking the required amount of tokens.
  - Receive information about available requests for tasks they are registered for, including the task requirements, input data, proposed payment, and relevant timing information.
  - Accept or decline participation in available inference requests.
  - Submit a prediction that conforms to the task's required output format for each request they accept.
  - Access information about their reputation, staked tokens, and rewards earned for each registered task.

- **Blockchain Network Responsibilities:**
  - Process user-authorized marketplace transactions related to inference request payments, provider staking and slashing, and provider rewards.
  - Execute the deterministic request state machine, including request creation, provider registration and per-task reputation, final aggregation, reputation updates, and reward calculation and distribution.
  - Make transaction status and transaction records available for verification.

2.3.3 Operating Context and Safety Significance

DAIIM is intended to be used as a web-based marketplace for requesting and evaluating AI inference results from multiple independent providers. Users submit inference requests through the system and receive aggregated predictions along with information about the contributing providers and the results they submitted. Blockchain technology is a required component of DAIIM, supporting transparent and verifiable records of marketplace activities, including provider stakes, payments, aggregated results, and reward distributions. DAIIM may be used in applications where reliable predictions are important, such as wildfire detection, where inaccurate or unavailable results could delay the identification of potential hazards. By combining predictions from multiple providers, the system aims to reduce dependence on any single provider and improve the reliability and availability of inference results. However, using multiple providers does not guarantee a correct result, as providers may submit inaccurate predictions or share similar sources of error. Although DAIIM does not directly control physical systems or make decisions on behalf of its users, its results may inform safety-critical applications where incorrect predictions could have serious consequences. Users are therefore responsible for validating aggregated results and determining whether they are sufficiently reliable for their intended use. The system must also account

for risks such as provider collusion, unavailable providers, failed transactions, and incorrect aggregation or reputation calculations, as these may affect the trustworthiness of its results and marketplace operations.

2.4 User Characteristics

DAIIM is intended to support users with varying levels of technical expertise.

- **Task Requesters:** Task requesters should have a basic understanding of the inference task they wish to perform and be able to provide the required input data. They should be able to use a modern web browser and interpret the inference results presented by the system. They are not expected to understand the internal aggregation, reputation, or reward algorithms.
- **Inference Providers:** Inference providers should have sufficient technical knowledge and computational resources to execute the inference tasks they accept. They should understand the requirements for submitting predictions in a format supported by the marketplace.

The user interface should minimize the technical knowledge required to submit requests and interpret results, while provider participation may require greater technical expertise.

2.5 System Constraints

- **Web3 browser compatibility:** The user-facing application must be accessed through a web browser that supports authentication with either an embedded wallet or a compatible Web3 wallet extension, and authorization of blockchain transactions through the selected wallet.
- **Inference type and data compatibility:** The system is restricted to regression-based inference for the initial project. Inference requests must provide input data in a supported format, and provider predictions must conform to the expected numerical output format so that results can be validated and aggregated consistently.
- **Blockchain integration:** Marketplace operations that require on-chain execution must be compatible with the blockchain network selected for the project. The system must account for possible transaction delays or failures.

3 Stakeholder Requirements

This section lists the stakeholder requirements for DAIIM, grouped by the stakeholders described in Section 1.3. These requirements are technology-independent and are stated from the point of view of the stakeholders.

Task Requesters

SR1 Task requesters shall be able to define their own machine learning task, including the type of input the task takes and the type of output it produces.

SR2 Task requesters shall be able to submit an inference request for a defined task and receive a result for that request.

SR3 Task requesters shall receive inference results that are reliable. Each result shall be based on predictions from multiple independent inference providers, so that no single faulty or dishonest provider can determine the result.

SR4 Task requesters shall receive results for their inference requests even when some inference providers are unavailable.

SR5 Task requesters shall receive information about the quality of each inference result, so that they can judge how much to trust it.

SR6 Task requesters shall be able to obtain inferences without relying on a single organization to coordinate the request, supply the prediction, or control the result.

SR7 Task requesters shall be able to see the actions and payments they authorize, and verify that those actions and payments were carried out.

SR8 Task requesters shall be able to define tasks, submit inference requests, and receive results without knowledge of decentralized systems or their underlying infrastructure.

Inference Providers

SR9 Inference providers shall be able to offer predictions from their own models for tasks defined by task requesters.

SR10 Inference providers shall be able to earn a profit by supplying accurate predictions to the network.

SR11 Inference providers shall be rewarded according to the accuracy of their predictions, so that supplying honest, accurate predictions is more profitable than supplying dishonest or careless ones.

SR12 Inference providers shall receive payment for their predictions without manual intervention from the task requester or any third party.

Secondary Stakeholders

SR13 Personal information and sensitive data handled by DAIIM, such as inference inputs and training data, shall only be accessible to the parties that need it to fulfill an inference request.

SR14 Personal information and sensitive data handled by DAIIM shall be protected from unauthorized access while it is transferred and while it is stored.

SR15 Personal information and sensitive data handled by DAIIM shall not be retained longer than necessary to fulfill the inference request it belongs to.

4 User Personas

4.1 Persona 1: Emergency Management Coordinator (Task Requester)

- Name: Priya Nair
- Description: Priya is 41 and coordinates emergency management for a mid-sized town on a flood-prone river, in a region that also has an annual wildfire season. She decides when to issue flood warnings and evacuation orders, and when to pre-position firefighting resources. She currently relies on a single forecasting service, and has to justify every decision to the town council.
- Goals: Receive river-level forecasts 24 to 48 hours ahead, and wildfire risk estimates during fire season, that combine predictions from multiple independent sources. See how much the sources agree, so she knows how much to trust each forecast. Keep costs predictable and within the town's budget.
- Frustrations: Her current service is a black box: it gives one number with no indication of confidence, and she has no alternative when it is wrong or unavailable. A missed flood forecast the previous spring left the town unprepared. She has no interest in learning about cryptocurrency or managing digital wallets.
- Technical Ability: Comfortable with spreadsheets, web dashboards, and mapping tools. No programming experience and no experience with blockchain or Web3 applications.

4.2 Persona 2: Independent Stock Forecaster (Inference Provider)

- Name: Daniel Okafor
- Description: Daniel is 29 and works as a software developer. In his spare time he builds machine learning models that predict next-day stock prices from historical market data.

His models perform well in backtests, but he has no way to earn money from them other than trading with his own savings.

- Goals: Earn a steady side income by supplying predictions from his models. Get paid automatically and promptly for good predictions. Build a visible track record that shows his models are reliable. Keep the cost of submitting each prediction low enough that participating is worthwhile.
- Frustrations: Selling predictions usually requires connections at a fund. Trading his own money exposes him to large losses. Prediction platforms he has tried were unclear about how payouts were calculated.
- Technical Ability: Strong Python programmer who regularly uses machine learning libraries. Owns a cryptocurrency wallet and has used decentralized finance applications.

4.3 Persona 3: Hydrology Researcher (Inference Provider)

- Name: Dr. Elena Varga
- Description: Elena is 34 and is a postdoctoral researcher in hydrology. She specializes in forecasting streamflow in mountain rivers fed by melting snow, where standard forecasting models often perform poorly during the spring melt. Her models currently only appear in academic papers.
- Goals: Earn supplementary income from her expertise by supplying predictions for tasks in her field, such as river-level and streamflow forecasts. Build a reputation for her models outside academia. See her research help communities prepare for floods.
- Frustrations: Consulting contracts are slow to arrange and rarely available to early-career researchers. Her models are most valuable during unusual events, when they disagree with standard models, and she worries about being penalized for disagreeing with the majority even when she is right.
- Technical Ability: Expert in statistical modeling, with strong Python and R skills. Has never used a cryptocurrency wallet or a blockchain application.

5 Use Cases

5.1 UC1: Sign in to the web application.

Primary Actor User (requester or provider)

Preconditions

- User has access to either a supported account for embedded-wallet authentication or a compatible Web3 wallet extension.

Main Success Scenario

1. User opens the DAIIM web application in a browser.
2. User clicks the "Sign in" button.
3. System displays the supported wallet authentication options.
4. User selects either an embedded wallet or a compatible Web3 wallet extension.
5. For an embedded wallet, the user authenticates using a supported social account, email, or phone number. For a wallet extension, the user connects the wallet and signs a verification message.
6. System verifies the authentication response and the public key linked to the user's wallet.
7. System authenticates the user and displays their profile linked with their wallet.

Alternative Flows

- 5a. User cancels authentication or rejects the wallet connection or signature: System cancels sign-in and redirects the user to the public page.
- 5b. User provides incorrect credentials or the wallet connection fails: System displays an error message and prompts the user to try signing in again or choose another authentication option.
- 6a. System fails to verify the authentication response or public key: System displays an error message and prompts the user to try signing in again.

Postconditions

- The requester is signed in to the DAIIM web application and can access their profile and other features of the application.

Related Requirements: FR1, FR2, NFR8, NFR3, NFR5, NFR17, NFR18, NFR.SA.2, NFR.SA.3.

5.2 UC2: Sign out of the web application.

Primary Actor User (requester or provider)

Preconditions

- User is signed in to the DAIIM web application.

Main Success Scenario

1. User clicks the "Sign out" button.

2. System checks for pending transactions or actions that need to be completed before signing out.

3. System confirms that there are no pending transactions or actions.

4. System ends the user's session.

5. System logs the user out and displays the public page.

Alternative Flows

- 2a. There are pending transactions or actions: System displays a warning message to the user indicating that they have pending transactions.

Postconditions

- The user is signed out of the DAIIM web application and returns to the public page where they cannot access their profile or other features of the application.

Related Requirements: FR56, NFR4, NFR6.

5.3 UC3: Add funds to Web3 Wallet

Primary Actor User (provider or requester)

Preconditions

- User (provider or requester) has a supported Web3 wallet authorized.

Main Success Scenario

1. User navigates to the "Add Funds" section of the DAIIM web application.

2. User interacts with embedded wallet infrastructure to initiate the process of adding funds to their Web3 wallet.

3. System displays the available funding options (e.g., credit/debit card, bank transfer, cryptocurrency transfer).

4. User selects their preferred funding option and provides the necessary information (e.g., card details, bank account information, or cryptocurrency wallet address).

5. System processes the funding request and verifies the transaction.

6. System updates the user's Web3 wallet balance and system displays a confirmation message indicating that the funds have been successfully added.

Alternative Flows

- 4a. User cancels the funding process: System cancels the funding process and returns the user to the "Add Funds" section.
- 5a. System fails to verify the transaction: System displays an error message and prompts the user to try adding funds again or choose another funding option.
- 5b. User provides incorrect or incomplete information: System displays an error message and prompts the user to correct the information and try again.
- 5c. User's funding method is declined or insufficient funds: System displays an error message and prompts the user to try a different funding method or check their account balance.

Postconditions

- The user's Web3 wallet balance is updated.
- The transaction is logged on the blockchain.

Related Requirements: FR4, FR55, FR56, FR57, FR.SA.6, NFR8, NFR7, NFR5, NFR6, NFR.SA.1, NFR.SA.2, NFR.SA.3.

5.4 UC4: Browse marketplace information

Primary Actor User (requester or provider)

Preconditions

- User has opened the DAIIM web application and is on the public page.

Main Success Scenario

1. User navigates to the "Browse Marketplace" section of the DAIIM web application.
2. System displays a list of available items (providers, reputations, tasks, etc.) in the marketplace.

Alternative Flows

- 2a. System cannot load list of items: System displays error message indicating items cannot be loaded and to try again later.

Postconditions

- User has successfully browsed the marketplace information.

Related Requirements: FR3, FR10, NFR4, NFR13, NFR5, NFR17.

5.5 UC5: Create an inference task

Primary Actor Authenticated user

Preconditions

- User is authenticated and logged in to the DAIIM web application.

Main Success Scenario

1. User navigates to the "Create Task" section of the DAIIM web application.
2. User fills out the task creation form with the required information.
3. System validates the information provided.
4. System requests confirmation of action from user.
5. User confirms and signs the transaction.
6. System saves the new task and displays a confirmation message.

Alternative Flows

- 2a. User cancels the task creation process: System discards the entered information and returns the user to the previous page.
- 3a. System detects missing or invalid information: System displays an error message and prompts the user to correct the information before proceeding.

Postconditions

- The new inference task is created in the on-chain program

Related Requirements: FR5, FR6, FR7, FR8, FR9, FR4, FR55, FR.SA.2, FR.SA.4, NFR5, NFR7, NFR4.

5.6 UC6: Register as an inference provider for a task

Primary Actor Provider

Preconditions

- Provider has an authenticated wallet and a model that can perform inference for a task.
- The task exists.

Main Success Scenario

1. Provider fills out the provider template and connects via the SDK.

2. System verifies that the provider is properly connected.
3. System verifies that the provider is not already registered for the task.
4. System verifies that the provider's wallet holds at least the task's minimum stake.
5. System requests that the provider consent to the staking and reputation conditions, the public display of the provider's name and reputation, the privacy agreement, and informs the provider that on-chain data is permanent.
6. Provider consents.
7. System requests the stake from the provider's wallet, explaining the action and the amount required.
8. Provider confirms and signs the stake transaction.
9. System verifies that the stake meets the task's minimum, locks it in escrow, assigns the base reputation for the task, and adds the provider to the list of registered providers for the task.
10. System displays the transaction status until it is confirmed and returns the transaction identifier to the provider.

Alternative Flows

- 1a. The provider template is incomplete or invalid: the SDK rejects it and names the missing or invalid field.
- 2a. The provider is not connected, the session has expired, or the wallet is on the wrong cluster: the system displays an error with the cause and a suggested fix, and no funds move.
- 3a. The provider is already registered for the task: the system displays an error and the provider's current registration, and the use case ends.
- 3b. The provider was registered before and unregistered: registration proceeds, and the reputation starts at the base reputation regardless of the earlier score.
- 4a. The wallet balance is below the minimum stake, or there is not enough SOL for the network fee: the system displays an error showing the shortfall for each, and the use case ends.
- 6a. The provider does not consent: the system explains that registration requires consent, nothing is signed or recorded, and the use case ends. The provider may start again.

- 8a. The provider declines to sign the stake transaction: no stake is locked, the provider is not registered, and the use case ends. The provider may start again.
- 9a. The system detects that the stake does not meet the task's minimum: the transaction is rejected, and no stake is locked.
- 9b. The network is unavailable or the transaction fails: no stake is locked and the provider is not registered. The system displays an error stating that no funds moved, and a retry does not create a duplicate registration.

Postconditions

- The task's minimum stake is locked in escrow.
- The provider is registered for the task with the base reputation, appears in the task's list of registered providers, and can accept inference requests and submit predictions for it.
- The registration, stake, and provider's consent are recorded and the provider has the transaction identifier.

Related Requirements: FR11, FR12, FR13, FR4, FR55, FR56, FR57, FR58, FR.SA.1, FR.SA.2, FR.SA.3, FR.SA.4, FR.SA.6, FR.SA.7, NFR5, NFR6, NFR7, NFR13, NFR.SA.2, NFR.SA.1.

5.7 UC7: Submit an inference request for a task

Primary Actor Requester

Preconditions

- Requester is logged in and authenticated and there are at least two providers available and registered for the task.

Main Success Scenario

1. Requester navigates to the "Submit Inference Request" section of the DAIIM web application.
2. Requester selects the desired task from the list of available tasks.
3. Requester provides the required input data for the inference request.
4. System validates the input data and checks for any errors or inconsistencies.
5. System displays token cost and estimated time for the inference request.
6. Requester confirms the submission of the inference request and authorizes the payment.

7. System processes the payment, deducts the appropriate amount of tokens from the requester's wallet, and holds the funds in escrow.
8. System processes the inference request and stores data in a secure location.
9. System sends request to the registered inference providers for the selected task.
10. System displays updated status of the inference request to the requester.

Alternative Flows

- 2a. Desired task is not available: Requester selects option to create a new inference task.
- 4a. Input data is invalid or incomplete: System displays an error message and prompts the requester to correct the input data before proceeding.
- 6a. Requester cancels the submission: System cancels the inference request and returns the requester to the previous page.
- 7a. Payment fails or is declined: System displays an error message and prompts the requester to try again or choose another payment method.
- 8a. System fails to process the inference request: System displays an error message and prompts the requester to try again later. Funds are returned to the requester.
- 8b. System fails to store data in a secure location: System displays an error message and prompts the requester to try again later, and removes input data from records. Funds are returned to the requester.
- 9a. Insufficient registered inference providers accept the request: System displays a message indicating that the inference request cannot be processed at this time and suggests trying again later or selecting a different task, funds are returned to the requester.

Postconditions

- The inference request is submitted and requester awaits result.

Related Requirements: FR17, FR18, FR19, FR20, FR23, FR38, FR4, FR55, FR56, FR59, FR60, FR61, FR.SA.1, FR.SA.2, FR.SA.7, FR.SA.8, FR.SA.6, NFR5, NFR6, NFR7, NFR9, NFR10, NFR16, NFR14, NFR.SA.3.

5.8 UC8: Accept an inference request for a task

Primary Actor Provider

Preconditions

- Provider is logged in, connected, and authenticated for the task.

Main Success Scenario

1. System broadcasts the inference request to all registered providers for the task, indicating the request, expected payout, stake amount, and deadline.
2. Provider receives the inference request and reviews the details.
3. Provider accepts the inference request.
4. System verifies that the provider has sufficient funds to stake to accept the request.
5. System prompts provider to confirm transaction and stake amount.
6. Provider confirms and signs the transaction.
7. System locks the stake in escrow and records the provider's acceptance of the inference request.

Alternative Flows

- 3a. Provider declines the inference request: System records the provider's decision and does not lock any stake.
- 4a. Provider does not have sufficient funds: System sends an error message detailing insufficient funds as the cause of rejection. Provider is not registered as an accepted provider for the request.
- 5a. Provider cancels the acceptance process: System cancels the acceptance.

Postconditions

- The provider is registered as an accepted provider for the inference request, and the stake is locked in escrow.

Related Requirements: FR20, FR21, FR22, FR24, FR4, FR55, FR.SA.2, FR.SA.3, FR.SA.4, NFR5, NFR7, NFR.SA.2.

5.9 UC9: Commit and reveal a prediction for an inference request

Primary Actor Provider

Preconditions

- Provider is logged in, connected, and authenticated for the task.
- Provider has been registered as an accepted provider for the inference request.

Main Success Scenario

1. System provides the decryption key for the input data to the accepted provider.
2. Provider retrieves the input data from the decrypted secure location.
3. Provider processes the input data using their model to generate a prediction.
4. Provider generates a salt using a cryptographically secure random number generator.
5. Provider hashes the request ID, provider wallet, prediction, and salt together to create a commitment, then submits the commitment to the on-chain program during the commit phase.
6. The program records the commitment and confirms it to the provider.
7. During the reveal phase, provider submits the prediction and salt directly to the on-chain program.
8. The on-chain program hashes the request ID, provider wallet address, revealed prediction, and salt, then compares the resulting hash with that provider's stored commitment.
9. If the hashes match and the prediction conforms to the task's output schema, the program records the valid revealed prediction; otherwise, it rejects the reveal.

Alternative Flows

- 4a. Provider does not submit a commitment by the commit deadline: provider is marked as a no-commit provider and slashed under FR49.
- 4b. Provider commits but does not reveal by the reveal deadline: provider is marked as a non-revealer and slashed under FR50.
- 5a. The reveal does not match the commitment: the program rejects it, does not count it toward the minimum, and slashes the provider under FR48.
- 5b. The revealed prediction is invalid under the output schema: the program rejects it, does not count it toward the minimum, and slashes the provider under FR48.

Postconditions

- After a valid reveal, the prediction and provider information is recorded for the inference request.

Related Requirements: FR25, FR59, FR26, FR15, FR16, FR27, FR28, FR29, FR30, FR48, FR49, FR50, NFR11, NFR12, NFR9, NFR10, NFR5, NFR.SA.3.

5.10 UC10: Aggregate predictions and produce a final inference result

Primary Actor System

Preconditions

- The reveal phase has ended, and at least the task's minimum number of providers with non-zero reputation have submitted valid revealed predictions to the on-chain program.

Main Success Scenario

1. The on-chain program calculates and stores the final inference result from valid predictions of providers with non-zero reputation, using the weighted-median algorithm specified in Appendix A (Weighted-Median Aggregation Algorithm) and the snapshot defined in FR35.
2. System displays the final inference result in the requester's dashboard, along with the accuracy information and number of provider results used.
3. System removes the input data and predictions from the secure location to maintain privacy and data retention policies after a set retention time has elapsed.

Alternative Flows

- 1a. System fails to retrieve all submitted predictions due to issues with the storage or network: System retries the operation, ensuring the transaction is not processed twice.
- 2a. System encounters an error during the aggregation process: System cahnges request status to failed. Requester is refunded and providers are compensated according to the terms in the contract.

Postconditions

- Final result is successfully calculated and stored, and the result is displayed to the requester.

Related Requirements: FR31, FR35, FR36, FR37, FR38, FR58, FR61, FR.SA.1, NFR1, NFR2, NFR11, NFR14.

5.11 UC11: Update and maintain provider reputation

Primary Actor System

Preconditions

- The final inference result has been calculated.

Main Success Scenario

1. The on-chain program calculates the accuracy of each provider's prediction by comparing it to the final inference result produced using the weighted-median algorithm in Appendix A (Weighted-Median Aggregation Algorithm).
2. The on-chain program calculates and applies the reputation score update for each provider whose valid prediction was included in the aggregate, as specified in FR39.

Alternative Flows

- 1a. System encounters an error during the accuracy calculation or reputation update process: System retries the operation.
- 2a. System calculates a reputation score that is below the minimum threshold for the task: System informs the provider that their reputation score has fallen below the minimum threshold and the consequences, unregisters them from the task, and burns their staked tokens.
- 2b. Provider has been slashed for at least REPEAT_OFFENSE_COUNT of the last REPEAT_OFFENSE_WINDOW: Provider is notified of offense and concequences. Provider is unregistered from the task and tokens are burned.

Postconditions

- Provider reputations are updated based on the accuracy of their predictions, and any providers with reputations below the minimum threshold are unregistered from the task.

Related Requirements: FR30, FR39, FR48, FR53, FR58, FR.SA.10, NFR1, NFR2.

5.12 UC12: Distribute rewards to inference providers

Primary Actor System

Preconditions

- The final inference result has been calculated, and provider reputations have been updated.

Main Success Scenario

1. The on-chain program calculates and distributes rewards for eligible providers according to FR41.
2. The on-chain program automatically transfers rewards to each eligible provider's wallet using the escrowed payment for the request.
3. The system displays the confirmed on-chain transfers and updated balances to each provider.
4. System sends a confirmation message to each provider, indicating the amount of tokens received and their updated balance.

Alternative Flows

- 2a. Calculated reward distribution results in a provider receiving a negative or zero reward: System sets the provider's reward to zero and informs them of the outcome.
- 3a. Provider wallet is no longer valid or accessible: System keeps that provider's reward in escrow, records the transfer as unresolved, sets the request status to "Pending Transfer", and notifies the provider of the held reward and the pending transfer deadline. Transfers to the other providers continue as normal.
  - 3a1. The provider retries the transfer before the pending transfer deadline, once their wallet can receive it: System transfers the held reward and records the transfer as resolved. Use case resumes at step 4.
  - 3a2. The transfer is still unresolved at the pending transfer deadline: System burns the held reward and records the transfer as forfeited.

Postconditions

- All eligible providers have received their rewards or had their tokens slashed, and their balances have been updated accordingly. Any reward that could not be transferred by the pending transfer deadline has been burned.
- No tokens for the request remain in escrow, and the request status is "Completed".

Related Requirements: FR41, FR42–FR46, FR14, FR58, FR.SA.1, FR.SA.9, FR.SA.10, NFR2, NFR14, NFR7.

5.13 UC13: Cancel an inference request with too few participating providers

Primary Actor System

Preconditions

- The reveal phase has ended with fewer than the task's minimum number of providers with non-zero reputation having validly revealed predictions.

Main Success Scenario

1. The system identifies that fewer than the task's minimum number of providers with non-zero reputation have validly revealed predictions.
2. The system cancels the inference request. Under FR49 and FR50, it slashes accepted providers who did not commit and committed providers who did not reveal.
3. The system notifies the requester and accepted providers about the cancellation.
4. The system refunds the requester's payment.
5. The system confirms the refund of tokens and updated balances for each affected user.
6. The system updates the status of the inference request to "Cancelled" in the system records and removes associated data from storage.

Postconditions

- Request is cancelled, tokens are released from escrow and refunded, and affected users are updated.

Related Requirements: FR22, FR23, FR15, FR16, FR30, FR31, FR49, FR50, FR33, FR38, FR61, FR.SA.1, NFR14, NFR7.

5.14 UC14: Unregister as an inference provider for a task

Primary Actor Provider

Preconditions

- Provider is logged in, connected, and registered for a task.

Main Success Scenario

1. Provider requests to unregister from a task.
2. System verifies that the provider is registered for the task and has no pending inference requests.
3. System prompts the provider to confirm the unregistration and informs them of any consequences, such as loss of reputation or forfeiture of staked tokens.
4. Provider confirms the unregistration.
5. System removes the provider from the list of registered providers for the task and updates their profile accordingly.

Alternative Flows

- 1a. Provider is not registered for the task they have selected: System sends an error message indicating that the provider is not registered for the task and cannot unregister.
- 2a. Provider has pending inference requests for the task: System sends an error message indicating that the provider cannot unregister until all pending requests are completed or withdrawn.
- 4a. Provider does not confirm the unregistration: System cancels the unregistration process and maintains the provider's registration for the task.

Postconditions

- Provider is unregistered from the task, and their remaining stake is returned or burned according to FR54.

Related Requirements: FR13, FR54, FR12, FR14, FR.SA.10, NFR5, NFR7.

5.15 UC15: View transaction history and current balances

Primary Actor User (requester or provider)

Preconditions

- User is logged in and authenticated.

Main Success Scenario

1. User navigates to the "Transaction History" section of the DAIIM web application.
2. System retrieves the user's transaction history and current balances from the blockchain.
3. System displays the user's transaction history and current balances associated with their wallet in a user-friendly format.
4. User reviews the transaction history and current balances, including details such as transaction IDs, timestamps, amounts, and status.

Alternative Flows

- 2a. System fails to retrieve transaction history due to lack of history: System displays a message indicating that there is no transaction history available for the user.

Postconditions

- User has successfully viewed their transaction history and current balances.

Related Requirements: FR56, FR57, FR38, FR14, FR58, NFR4, NFR6.

5.16 UC16: Use in-app guidance to learn the system

Primary Actor User (authenticated or unauthenticated)

Preconditions

- User has the DAIIM browser application open.

Main Success Scenario

1. User opens a feature or begins a task in the DAIIM application.
2. System displays concise instructions and optional tooltips relevant to the current screen and task.
3. User follows the in-app guidance to complete the task.

Alternative Flows

- 2a. User dismisses a tooltip: the tooltip closes and the user continues using the application.

Postconditions

- User has completed the task with the help of clear in-app instructions and tooltips.

Related Requirements: NFR3, NFR4, NFR5, NFR17.

6 Functional Requirements

The functional requirements are grouped by the area of DAIIM they describe.

6.1 Accounts and Authorization

FR1: The system shall allow users to authenticate using either an embedded wallet or a compatible Web3 wallet extension and verify ownership of the associated wallet address using an authentication response from the selected wallet. The system shall associate each verified wallet address with the same user profile on every successful authentication, including after the user signs out. When a user chooses to create an embedded wallet, the system shall follow FR2.

FR2: When a user requests an embedded wallet and provides a supported social account, email address, or phone number and completes the required authentication, the system shall create an embedded wallet associated with the user and provide the wallet's public address and a creation confirmation. If the input is invalid or wallet creation fails, the system shall report the failure and shall not indicate that the wallet was created.

FR3: The system shall allow unauthenticated users to browse publicly available marketplace information, including registered providers, available models, and provider reputations.

FR4: The system shall require authenticated users to authorize (sign) blockchain transactions through their connected wallet before executing actions that transfer tokens or modify on-chain state.

6.2 Inference Tasks

FR5: The system shall allow authenticated users to create new inference tasks by providing the task definition.

FR6: Each inference task definition shall specify a name, problem description, inference category or type, input schema, and output schema. Where applicable, it shall also specify the metric(s) used to evaluate predictions. The task creator may optionally provide training data, or a reference to training data, for providers to use when training or adapting models for the task.

FR7: Each inference task definition shall specify the minimum and maximum number of participating providers required to fulfill an inference request for the task.

FR8: The system shall reject inference task definitions that specify a minimum number of participating providers less than 2.

FR9: Each inference task definition shall specify the length of the acceptance period for the task's inference requests. The acceptance period for each request shall end this length of time after the request is broadcast.

FR10: The system shall allow all users to browse existing inference tasks and view each task's definition, required minimum provider stake, and number of registered providers.

6.3 Provider Registration and Staking

FR11: The system shall allow authenticated users to register as inference providers for an inference task by staking the required minimum amount of tokens. Upon registration, the on-chain program shall create a provider entry for the provider's wallet address and store the provider's reputation and stake in that entry.

FR12: When a provider registers for an inference task, the system shall lock the required minimum stake for that task in escrow. The system shall return the provider's stake when the applicable conditions for releasing it are met.

FR13: The system shall allow registered inference providers to unregister from an inference task, provided they have not accepted any active inference requests for that task. Upon unregistration, the provider shall be removed from the list of registered providers for the task, and the provider's remaining stake shall be returned or burned according to FR54.

FR14: The system shall allow inference providers to view their reputation score, staked tokens, and rewards earned for each inference task they are registered for.

6.4 Inference Requests

FR15: The protocol shall define one system-wide COMMIT_PERIOD, used for every inference request and measured from the end of that request's acceptance period.

FR16: The protocol shall define one system-wide REVEAL_PERIOD, used for every inference request and measured from the end of that request's commit phase.

FR17: The system shall allow authenticated users to submit inference requests for an existing inference task. Each request shall include input data that conforms to the task's input schema, and the on-chain program shall create a unique on-chain request record for it. The on-chain program will not store the request's input data.

FR18: The system shall reject inference requests whose input data does not conform to the task's input schema.

FR19: When submitting an inference request, the system shall require the requester to choose a payment amount in tokens greater than the minimum and prompt them to authorize, through their connected wallet, a transaction transferring that amount of tokens into escrow.

FR20: After an inference request is submitted, the system shall broadcast its task definition, proposed payment amount, acceptance-period expiry time, commit deadline, and latest possible reveal deadline to every provider registered for the task. When the commit phase ends, the system shall broadcast the actual reveal deadline to every accepted provider.

6.5 Provider Acceptance and Predictions

FR21: Upon creation of an inference request, the on-chain program shall automatically begin the acceptance period, during which registered providers may accept or decline participation.

FR22: At the specified expiry time, the system shall end the acceptance period. All registered providers that have not accepted the request will be treated as non-participating. Inference request acceptances from inference providers outside of the acceptance period will not be considered.

FR23: If fewer than the task's minimum number of participating providers have accepted an inference request when the acceptance period ends, the system shall cancel the request and return the requester's escrowed payment.

FR24: The on-chain program shall reject an acceptance that would exceed the task's maximum number of participating providers. Once that maximum is reached, the acceptance period shall close and no further provider may accept the request.

FR25: Once the acceptance period has ended, the system shall make the request's input data, or information needed to access it, available to all providers that accepted the request during the acceptance period.

FR26: At acceptance-period expiry, the on-chain program shall begin the commit phase for accepted providers; only accepted providers may commit.

FR27: Each accepted provider shall submit its commitment directly to the on-chain program during the commit phase. The commitment shall be a cryptographic hash of the request ID, provider wallet address, prediction, and salt.

FR28: The on-chain program shall accept at most one commitment and one reveal from each provider for a given request.

FR29: During the reveal phase, each provider shall submit its prediction and salt directly to the on-chain program. The program shall recompute the commitment and accept the reveal only if it matches the stored commitment and the prediction conforms to the task output schema. Mismatched or non-conforming reveals shall be rejected and shall not count toward the minimum valid prediction count.

FR30: The commit phase shall end when all accepted providers have committed or the commit period expires, whichever occurs first. The reveal phase shall then begin and end when all committed providers have revealed or the reveal period expires, whichever occurs first.

FR31: At reveal-phase end, if fewer than the task's minimum number of valid revealed predictions are available, the system shall cancel the request and return the requester's escrowed payment.

FR32: When a provider accepts an inference request, the system shall reserve (MINIMUM_TASK_PROVIDE
1) × PROPOSED_PAYMENT_AMOUNT of the provider's stake for the task for that request, where MINIMUM_TASK_PROVIDERS is the task's minimum number of participating providers (FR7) and PROPOSED_PAYMENT_AMOUNT is the request's

per-provider compensation. Reserving stake shall not transfer any tokens. The system shall reject the acceptance if the provider's stake for the task, minus all of the provider's existing reservations for the task, minus the new reservation, would be less than MINIMUM_PROVIDER_STAKE.

FR33: When an inference request is completed or cancelled, the system shall release every reservation made for it under FR32, except for any amount paid out under FR34.

FR34: When an inference request is cancelled under FR31, the system shall pay PROPOSED_PAYMENT_AM to each provider that submitted a valid revealed prediction for the request. The total shall be taken in equal parts from the reservations of the accepting providers whose reveal was rejected under FR29, who did not commit, or who did not reveal. These compensation transfers do not replace slashing required by FR49, FR50, or FR48.

6.6 Aggregation and Results

FR35: The on-chain program shall aggregate only valid revealed predictions from providers whose reputation score at the end of the reveal phase, before any reputation updates for that request, is non-zero. For each included prediction, its weight shall be that reputation score divided by the sum of the reputation scores of all included providers. The on-chain program shall calculate and store the final numerical result using the weighted-median algorithm specified in Appendix A (Weighted-Median Aggregation Algorithm).

FR36: The system shall return the final aggregated result, time elapsed since submitting the request, and the number of provider predictions included in the aggregation to the user that submitted the inference request.

FR37: The system shall return, with each final aggregated result, the minimum and maximum of the predictions included in the aggregation.

FR38: The system shall allow requesters to view the status of each inference request they have submitted and, once available, its final aggregated result.

6.7 Reputation and Rewards

FR39: After the final aggregated result for an inference request is computed, the on-chain program shall update the reputation of every provider whose valid prediction was included in the aggregate under FR35, using the Confidence-Based Reputation Update Algorithm algorithm in Appendix A. The algorithm shall use the reputation scores at the end of the reveal phase, before any reputation updates for that request.

FR40: The system shall maintain each provider's reputation score within the inclusive range $[0, 100]$

FR41: After applying the reputation updates for an inference request, the on-chain program shall calculate and automatically distribute the request's reward among all providers whose predictions contributed to the final aggregated result and whose reputation did not decrease as a result of the request. The reward shares shall be proportional to those eligible providers' reputation scores after the updates have been applied.

FR42: If a reward transfer to a provider fails because the provider's wallet is invalid or cannot receive tokens, the system shall keep that provider's reward in escrow, record the transfer as unresolved, and set the request's status to Pending Transfer. Transfers to other providers and the requester's access to the final aggregated result shall not be affected.

FR43: When a reward transfer is recorded as unresolved, the system shall notify the provider and show the held reward amount and the request's pending transfer deadline on the provider's dashboard.

FR44: While a request is in Pending Transfer, the system shall allow a provider with an unresolved transfer to retry it. If the retried transfer succeeds, the system shall record the transfer as resolved.

FR45: Each inference request shall have a pending transfer deadline equal to the actual end of its reveal phase plus PENDING_TRANSFER_PERIOD. If a reward transfer is still unresolved at this deadline, the system shall burn the held reward and record the transfer as forfeited.

FR46: The system shall set an inference request's status to Completed only after every reward transfer for the request has either succeeded or been forfeited under FR45.

6.8 Penalties

FR47: The system shall slash a provider's stake only under the conditions specified in FR48, FR49, and FR50, and shall burn a provider's remaining stake only under the conditions specified in FR52, FR53, and FR54.

FR48: When the system rejects a provider's reveal for a commitment mismatch or a non-conforming prediction under FR29, the system shall slash SLASH_FRACTION of the provider's stake for the task.

FR49: When an accepted provider has not committed by the end of the commit phase, the system shall slash SLASH_FRACTION of the provider's stake for the task.

FR50: When a committed provider has not revealed by the end of the reveal phase, the system shall slash SLASH_FRACTION of the provider's stake for the task.

FR51: Whenever a provider's stake for an inference task falls below MINIMUM_PROVIDER_STAKE, the system shall automatically unregister the provider from that task.

FR52: When a provider has been slashed for at least REPEAT_OFFENSE_COUNT of the last REPEAT_OFFENSE_WINDOW inference requests they accepted for a task, the system shall unregister the provider from the task and burn their remaining stake for the task.

FR53: The system shall automatically unregister any inference provider whose reputation score reaches 0. Upon unregistering, the system shall burn the provider's remaining staked tokens for the associated inference task.

FR54: When a provider voluntarily unregisters from an inference task under FR13, let R be the provider's current reputation score and $R_{\text{exit}} > 0$ be the task's reputation threshold for a reduced stake return. If $R \geq R_{\text{exit}}$, the system shall return the entirety of the provider's remaining stake to their token account. If $0 \leq R < R_{\text{exit}}$, the system shall return $\frac{R}{R_{\text{exit}}}$ of their remaining stake to their token account and burn the remainder.

6.9 Transparency

FR55: Before prompting a user to authorize a transaction, the system shall display the action being performed and the amount of tokens being transferred.

FR56: The system shall display the status of each transaction a user has authorized until the transaction is confirmed or fails.

FR57: The system shall provide, for each transaction a user has authorized, the transaction identifier needed to look up the transaction on the blockchain.

FR58: The system shall create and maintain each inference request, the provider registry and per-task reputations, request payments, provider stakes, aggregated results, reputation updates, and reward distributions on the blockchain.

6.10 Data Handling

FR59: The system shall make a request's input data available only to the requester and the providers that accepted the request.

FR60: The system shall not store a request's input data on the blockchain in readable form.

FR61: The system shall delete a request's input data from any storage the system controls once the request has been completed or cancelled.

6.11 Safety Functional Requirements

FR.SA.1: The system shall not resubmit a transaction until it has confirmed that the original transaction was not processed and shall maintain logs of all transaction statuses and log verified transactions publicly on-chain.

FR.SA.2: The system shall verify if the wallet of a user has sufficient funds before requesting a signature and logging the transaction.

FR.SA.3: The on-chain programs shall reject any transaction that the wallet cannot cover at the time of execution, regardless of any balance check made earlier.

FR.SA.4: The system shall not submit any user-initiated transaction which is not signed by the authorized wallet.

FR.SA.5: The system shall wait for validity period to end and status to be checked before logging transaction as failed and attempting to reprocess.

FR.SA.6: When a transaction fails, the system shall send the error to the user and allow them to retry.

FR.SA.7: Before a transaction is submitted for signing, the system shall simulate it and verify that the resulting balance changes equal the amounts displayed to the user, and shall not submit it otherwise.

FR.SA.8: The amount charged to a requester for an inference request shall equal the payment amount the requester authorized.

FR.SA.9: The system shall validate each reward calculation before distributing it, and shall not distribute rewards whose total exceeds the reward for the request.

FR.SA.10: The system shall confirm provider status before distributing rewards/slashing tokens.

7 Data Requirements

7.1 Major Data Entities

- User Profiles: User accounts associated with verified wallet addresses, with information shown to the user on their dashboard.
- Model Providers: Network contributors identified by a wallet address, with a list of registered inference tasks and a reputation for each task.
- Inference Tasks: A task defined by its description and input and output formats, with a list of registered inference providers.

- Inference Requests: A consumer's request for an inference task, including input data, requester information, each provider's commitment and reveal status, and the final aggregated inference.
- Token Economics: Network-wide totals describing token supply, staking, escrow, burns, and reward distributions.
- Network Participation Metrics: Measurements summarizing model providers' participation in inference tasks and requests across the network.

7.2 Data Dictionary

Entity	Attribute	Type	Description
User Profile	Wallet addresses	List<Address>	Verified wallet addresses associated with the user profile
	Join date	Timestamp	Date and time the user profile was created
	Submitted requests	List<Request IDs>	Inference requests submitted by the user
	Registered tasks	List<Task IDs>	Inference tasks the user is registered to provide for
	Task reputation	List<Task ID, Number>	The user's reputation for each registered task
	Staked tokens	List<Task ID, Number of tokens>	Tokens the user has staked for each registered task
	Rewards earned	List<Task ID, Number of tokens>	Rewards earned by the user for each registered task
Model Provider	Wallet address	Address	Provider's wallet
	Registered tasks	Task IDs	Tasks the provider joined
	Task reputations	List<Number>	List of this Provider's reputation for registered tasks. Ranging from 0 to 100
	Amount Staked	List<Integer>	List of staked tokens per registerd task
	Slashing Offenses	List<Task ID, Request ID>	List of inference requests for which the provider was slashed
Inference Task	Task ID	Identifier	Unique task identifier
	Name and type	String	Task name and category

	Description	String	Problem addressed by the task
	Input schema	JSON	Valid request input format
	Output schema	JSON	Valid prediction format
	Minimum providers	Integer	Required participants per request
	Acceptance period	Duration	Time allowed for providers to accept a request
	Reputation tolerance	Number	Threshold for reputation changes
	Baseline reputation	Number	Reputation assigned to a provider when they register for the task
	Exit reputation threshold (R_{exit})	Number	Positive reputation threshold for determining the stake returned upon voluntary unregistration
	Minimum stake	Number	Stake required for registration
Inference Request	Request ID	Identifier	Unique request identifier
	Task ID	Identifier	Requested inference task
	Requester wallet	Address	Wallet of the requester
	Input data	JSON	Data submitted for inference
	Cost	Number	Offered payment
	Status	Status	Current request state and phase. Phase statuses include Acceptance, Commit, Reveal, and Finalization. Final states are Completed, Failed, and Cancelled. Pending Transfer means at least one reward transfer is unresolved
	Unresolved transfers	List	Provider wallets and reward amounts held in escrow until transferred or burned
	Provider commitments	Provider-to-hash list	Commitment digest submitted by each accepted provider
	Commit times	Provider-to-timestamp list	Time each commitment was recorded
	Commit statuses	Provider-to-status list	Whether each accepted provider committed before the commit phase ended
	Revealed predictions	Provider-to-prediction list	Predictions revealed by providers; only schema-valid, matching reveals are used in aggregation

	Reveal times	Provider-to-timestamp list	Time each reveal was submitted
	Reveal statuses	Provider-to-status list	Whether each reveal was valid, mismatched, invalid, or missing
	Aggregated result	Output Type	Final inference result
	Submission time	Timestamp	Request creation time
Token Economics	Total minted	Number of tokens	Tokens minted across the network
	Total staked	Number of tokens	Tokens staked by providers
	Request escrow	Number of tokens	Tokens escrowed for inference requests
	Total burned	Number of tokens	Tokens permanently removed from supply
	Rewards distributed	Number of tokens	Tokens paid to providers as rewards

Table 2: Data Dictionary

7.3 Privacy and Data Retention

DAIIM will use a semi-private approach for inference request input data. Inputs will be encrypted and made accessible only to the requester and providers that accept the request. By submitting a request, the requester consents to the input being shared with those providers and should not submit data they consider fully private or confidential. Input data will be stored only as long as needed to fulfill the request and will be deleted from storage controlled by the system once the request is completed or cancelled, following established data-minimization and retention practices. The system will not store readable input data on the blockchain. DAIIM's handling of personal data must follow applicable privacy and data-protection laws for its deployment and users; the specific obligations will depend on the relevant jurisdictions.

Other data, including wallet addresses, provider reputations, task definitions, request status and results, and token-economics records, will be collected and retained only as needed to operate the marketplace, provide its features, and meet applicable legal obligations. Information needed for public marketplace and blockchain functions may be visible to other users; users should be informed that blockchain records may be permanent and cannot necessarily be deleted. Data stored off-chain will be protected against unauthorized access and encrypted in transit and at rest where appropriate.

8 Non-Functional Requirements

8.1 Accuracy

NFR1: Simulation tests shall verify that the system's aggregate results match the weighted-median algorithm specified in Appendix A (Weighted-Median Aggregation Algorithm) for the tested predictions and reputation weights, including the worked examples in that appendix.

NFR2: For all predictions, reputation scores, and token amounts within their declared ranges, the aggregation, reputation, and reward calculations shall produce no overflow or underflow. Any arithmetic operation that would overflow shall cause the transaction to fail without changing on-chain state.

8.2 Usability

NFR3: A new requester with experience using a web browser but no prior knowledge of blockchain or machine learning shall be able to browse tasks, submit an inference request, and find its status within 5 minutes using clear in-app instructions and tooltips, verified through usability testing.

NFR4: All features on the web application shall be accessible within 2 clicks from the home page, verified through usability testing.

NFR5: Every user-facing error shall be visible and include a name, description, and suggested fix, verified through usability testing.

NFR6: Long operations shall always show progress status, verified via testing.

NFR7: Confirmation messages written in plain language shall be displayed after every transaction, indicating what has been recorded and if balances have been updated, verified via testing.

NFR8: Requesters shall be able to access or create an embedded wallet using supported social accounts, email, or phone numbers without needing prior knowledge of Web3 wallets, verified through user testing.

8.3 Security

NFR9: The system shall follow PIPEDA guidelines such that personal information and sensitive data handled by DAIIM, such as inference inputs, shall be encrypted while it is transferred and while it is stored by the system, so that it is protected from unauthorized access, verified through security audits.

NFR10: Input data shall be encrypted according to industry standard.

NFR11: No individual prediction shall be readable from on-chain or backend data before the commit phase ends. A simulation test shall attempt to read predictions from both sources before that point.

NFR12: The system shall use a salt of at least 256 bits using a cryptographically secure random number generator and persist it with the prediction until reveal completes, verified by inspection and simulation testing.

NFR13: Public pages shall display no personal information beyond what providers consented to, in accordance to PIPEDA.

8.4 Reliability and Availability

NFR14: An inference request shall reach a final state (Completed, Failed, Cancelled) and all escrowed tokens shall be deposited, slashed, or burned before the final deadline for the request has elapsed, verified through simulation. The final deadline is the request's pending transfer deadline (FR45); its latest possible time is the acceptance-period expiry plus COMMIT_PERIOD, REVEAL_PERIOD, and PENDING_TRANSFER_PERIOD.

8.5 Maintainability

NFR15: Code shall follow the standards in the Development Plan, enforced by linting.

8.6 Scalability

NFR16: The system shall support multiple concurrent inference requests without degradation to performance beyond `PERFORMANCE_DEGRADATION_TOLERANCE`.

8.7 Portability and Installability

NFR17: The system shall be fully operational on current versions of modern browsers with an internet connection.

NFR18: For users who choose embedded-wallet authentication, the web client shall not require software installation beyond a supported browser; users shall be able to access or create an embedded wallet using a supported social account, email, or phone number.

8.8 Safety

NFR.SA.1: The system shall send readable error messages to users within 2 seconds of a detected failure.

NFR.SA.2: All signing shall happen within the user's wallet and private information shall not be logged on-chain.

NFR.SA.3: All communication between DAIIM components, and between users and DAIIM, shall use TLS with certificate validation enabled to ensure data protection.

NFR.SA.4: The system shall scan third-party dependencies for known security vulnerabilities, verified through testing.

9 AI and Machine Learning Requirements

This section describes how DAIIM supports machine-learning inference, evaluates prediction quality, and communicates the limitations of its results. The project scope focuses on regression tasks: each task defines its input and output schemas, providers submit numerical predictions, and the system combines those predictions into a reputation-weighted result.

9.1 Evaluation Metrics

For regression tasks with available ground-truth values, DAIIM will evaluate predictions using mean absolute error (MAE), calculated as the arithmetic mean of the absolute differences between predicted and ground-truth values across evaluated requests:

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|,$$

where n is the number of evaluated requests, $\hat{y}_i$ is a predicted value, and y_i is its ground-truth value. MAE will be reported in the units of the task's output. Ground truth may not exist or may become available only later. DAIIM will calculate and display MAE for available predictions and aggregated results.

9.2 Known Limitations

Commit-reveal prevents copying from visible earlier submissions, but does not prevent providers from colluding or sharing predictions off protocol. The system aggregates provider predictions but does not independently verify that the final result is correct. Decentralization, provider reputation, and staking are intended to encourage reliable predictions, but the system cannot guarantee that a result is accurate or suitable for its intended use.

10 Requirements Traceability Matrices

Stakeholder Req	Use Case	Functional Req
SR1	UC5	FR5, FR6, FR7, FR8, FR9
SR2	UC7, UC10	FR17, FR18, FR19, FR20, FR36, FR38, FR.SA.8
SR3	UC10, UC11, UC13, UC9	FR7, FR8, FR23, FR27, FR28, FR29, FR30, FR31, FR35, FR39, FR40
SR4	UC8, UC9, UC13	FR21, FR22, FR24, FR23, FR15, FR16, FR30, FR31, FR.SA.5, FR.SA.6
SR5	UC10	FR36, FR37
SR6	UC8, UC9, UC10	FR20, FR27, FR29, FR30, FR35, FR58
SR7	UC3, UC7, UC15, UC6	FR4, FR55, FR56, FR57, FR58, FR.SA.1, FR.SA.2, FR.SA.3, FR.SA.4, FR.SA.7, FR.SA.8
SR8	UC1, UC2, UC4, UC16	FR1, FR3, FR10, FR.SA.2, FR.SA.6
SR9	UC4, UC6, UC8, UC9, UC14	FR10, FR11, FR12, FR13, FR21, FR25, FR26, FR27, FR29, FR30
SR10	UC12	FR41, FR42–FR44, FR14
SR11	UC11, UC12	FR39–FR41, FR47–FR53, FR.SA.9, FR.SA.10
SR12	UC12	FR12, FR41, FR42–FR46, FR58, FR.SA.9
SR13	UC7, UC8	FR25, FR59, FR60
SR14	UC7	FR60
SR15	UC7, UC13	FR61

Table 3: Traceability Between Stakeholder Requirements, Use Cases, and Functional Requirements

Functional Req	Nonfunctional Req
FR1	NFR8, NFR3, NFR18, NFR.SA.2
FR2	NFR8, NFR18
FR4	NFR.SA.2

FR3	NFR4, NFR13, NFR3
FR15–FR16	NFR14
FR10	NFR4, NFR13
FR12	NFR14
FR14	NFR4
FR17	NFR16
FR18	NFR5, NFR.SA.1
FR20	NFR16
FR23	NFR14
FR25	NFR10, NFR.SA.3
FR27–FR28	NFR11, NFR12
FR29	NFR5, NFR.SA.1
FR30	NFR11, NFR14
FR31	NFR14
FR35	NFR1, NFR2
FR38	NFR6, NFR4
FR39, FR40	NFR1, NFR2
FR41	NFR2, NFR14
FR42–FR46	NFR14
FR47–FR53	NFR14, NFR2
FR56	NFR6, NFR7, NFR5, NFR.SA.1
FR57	NFR7
FR59	NFR9, NFR10, NFR.SA.3
FR60	NFR9, NFR10
FR.SA.1	NFR14, NFR7
FR.SA.2	NFR5
FR.SA.3	NFR14
FR.SA.4	NFR.SA.2
FR.SA.5	NFR6, NFR14
FR.SA.6	NFR5, NFR.SA.1
FR.SA.7	NFR7, NFR2
FR.SA.8	NFR7, NFR2
FR.SA.9	NFR2, NFR14
FR.SA.10	NFR14

Table 4: Traceability Between Functional and Nonfunctional Requirements

11 Rationale

11.1 On-Chain Execution

Every inference task, inference request, the provider registry and per-task reputations, final aggregation, reputation updates, reward calculation and distribution are all executed and / or stored on the public blockchain. (FR17, FR11, FR35, FR39, FR41, and FR58). The backend is used for encrypted input delivery, transaction handling, validating transactions, user notifications and several other important functionalities, but it is not trusted to decide any state changes to the marketplace.

This boundary is necessary for decentralization, verifiability and the trustlessness of the system. If a backend controlled state changes, it could refuse to aggregate a request, omit a provider's prediction, delay a reputation update, or withhold a reward. Unlike a conventional piece of software, which runs on servers controlled by one organization, an on-chain program runs on a public blockchain network. No single entity can stop it, change its rules, or decide who may use it. Therefore, within DAIIM, any user can start an inference request, and the program itself checks that the required conditions are met, so the outcome does not depend on any central authority.

11.2 Provider Staking

Inference providers must stake a minimum amount of tokens to register for an inference task (FR11). The stake is locked in escrow by the system's on-chain logic, rather than held by any person, and can only be returned, slashed, or burned according to the rules in FR12, FR47, FR13, and FR53. Requiring a stake serves three purposes:

- **Resistance to fake accounts.** The final result is a reputation-weighted aggregate of provider predictions (FR35), so anyone who controls enough providers for a task can control its results. Without a stake, an attacker could register many providers at no cost, and a provider with a low reputation could abandon their account and register a new one. Requiring a stake for every registration makes each additional provider cost real tokens, which makes gaining control of a task's results expensive.

- **Penalties for misbehaviour.** A loss of reputation alone may not deter a provider who has nothing else at risk. A stake allows the system to impose a financial penalty by slashing it (FR47), or burning it when a provider's reputation falls below the minimum threshold (FR53).

- **Commitment to accepted requests.** If accepting an inference request had no cost, providers could accept every request and respond only when convenient, causing requests to be cancelled for having too few predictions (FR31). Having a stake at risk gives providers a reason to submit a prediction for every request they accept.

A provider's stake does not determine the weight of their predictions; reputation does (FR35). This prevents providers from gaining influence over results through wealth alone, so influence must be earned through a history of predictions that agree with the network.

The minimum stake must balance two concerns. It must be high enough that registering many fake providers is expensive, but low enough that honest new providers, who have not yet earned any rewards, are not discouraged from joining.

11.3 Voluntary Unregistration and Stake Return

Because DAIIM is permissionless, there is no viable way to keep track of individual users and determine if a given entity has multiple accounts. Unlike a conventional system that may identify users through information such as a phone number, email address, or IP address, DAIIM identifies providers by wallet address, and anyone can create additional wallet addresses at no cost.

This makes reputation resets possible even when the same-wallet registration history is retained: a provider can register a new wallet and receive the baseline reputation specified in UC6. A provider can also finish any active requests, voluntarily unregister, and re-register after their reputation has decreased. Valid but poor predictions can decrease reputation without triggering stake slashing under FR48 or FR49 and FR50; the provider can reuse any returned stake for a new registration. Under FR54, the fraction of a provider's remaining stake returned on voluntary unregistration depends on their reputation score relative to the task's exit threshold R_{exit}. This is intended to discourage providers from unregistering and re-registering solely to restore their reputation to its baseline value, while still allowing providers to leave a task and recover some or all of their stake.

The exit threshold R_{exit} must be chosen intelligiently to prevent re-registration attacks while still remaining financially attractive to participate. If R_{exit} is too high, providers whose predictions have been poor may lose a substantial part of their stake when they decide to stop participating, even if their poor performance was not intentional. If it is too low, a provider may retain most of their stake after a meaningful reputation decline and have little financial reason not to reset their reputation by leaving and re-registering. The task's R_{exit} should be chosen to allow providers to recognize that a task is not a good fit and exit without excessive financial loss, while making reputation resets through repeated unregistration and re-registration costly enough to discourage continued abuse.

11.4 Unresolved Reward Transfers

A reward transfer can fail if a provider's wallet can no longer receive tokens. Holding the reward indefinitely would keep tokens in escrow forever and break NFR14. Instead, the request moves to Pending Transfer, the provider can retry until the pending transfer deadline, and any reward still unclaimed is then burned (FR42–FR46). This gives every request a fixed final deadline that is known when the request is submitted: the acceptance-period expiry, plus COMMIT_PERIOD and REVEAL_PERIOD, plus PENDING_TRANSFER_PERIOD.

The requester still receives the final result during Pending Transfer, since a provider's wallet problem has nothing to do with the result. The request is not marked Completed until every transfer is settled, so its status always shows whether tokens remain in escrow. The unclaimed reward is burned rather than refunded, because the requester has already received the service they paid for, and rather than redistributed, because that would change other providers' rewards after they were paid.

11.5 Protocol-Wide Commit and Reveal Periods

The commit and reveal durations are protocol-wide constants rather than task settings. This keeps phase timing consistent for providers across requests and avoids making timing a task-definition choice that could be changed to favor or disadvantage participants. Each request derives its deadlines from the end of its acceptance period and the end of its commit phase; if a phase ends early, the corresponding next deadline is calculated from that actual phase end.

11.6 Commit-Reveal Privacy

Providers commit directly to the on-chain program, which records only a hash of the prediction and salt, and does not reveal the plaintext prediction until the commit phase ends. Buffering plaintext predictions in the backend would give the backend operator early access, censorship power, and exclusive knowledge of submissions. Each hash binds the request ID and provider wallet as well as the prediction and salt, preventing a provider from copying another provider's commitment or reusing it for another request. A salt of at least 256 bits prevents brute-force guessing across the small prediction space. Commit- reveal prevents copying based on observed submissions, but does not prevent off-protocol collusion or providers sharing predictions privately.

11.7 Request Broadcast and Collusion

Every registered provider is eligible to receive each request, and any registered provider may accept until the task's finite maximum is reached (FR20, FR21, FR24). The system does not invite a random subset of providers to each request.

Commit-reveal limits copying based on publicly visible submissions. It does not prevent off-protocol collusion, such as providers sharing predictions privately. The Hazard Analysis (H4.3) identified collusion as a risk: a coordinated group of providers could submit similar inaccurate predictions to skew the aggregated result, reputations, and rewards. Inviting only a random subset of providers to each request would reduce this risk, because colluders could not know in advance which of them would be invited to the same request. Their combined weight in the aggregate would also vary from request to request instead of being fixed. This mitigation is not part of the current scope, for three reasons:

- It requires a secure and verifiable source of randomness on Solana, so that providers cannot predict or influence who gets invited.

- Inviting fewer providers makes it less likely that a request reaches the task's minimum number of participating providers (FR7, FR23), so the minimum-provider rules would need to be reworked.
- With broadcast to all providers, a colluding group must make up a large share of the providers on a task to have much influence. Reputation-weighted aggregation (FR35) and staking already make this costly.

Instead, the proof of concept will simulate colluding groups of different sizes to measure how much they can affect results, and compare the broadcast design with random invitation. Random invitation is recorded as a possible future change in LC4.

12 Likely Changes

LC1: The set of supported AI inference models may expand beyond the regression model used in the initial proof of concept. The inference processing and aggregation components may need to be adapted to support additional model types.

LC2: The number and behaviour of participating inference providers may change over time as providers join or leave the marketplace. The system's aggregation, reputation, and reward mechanisms may need to accommodate different levels of provider participation.

LC3: The prediction aggregation, reputation, and reward mechanisms may be refined as the proof of concept is evaluated. These changes may be required to improve prediction reliability and address issues such as inaccurate predictions, malicious providers, or collusion.

LC4: Inference requests may in the future be sent to a randomly selected subset of registered providers instead of to every registered provider, to make premeditated collusion harder (see Request Broadcast and Collusion). This would replace the broadcast and eligibility rules in FR20 and FR21.

LC5: User requirements for the marketplace interface and the presentation of inference results may evolve as the system is tested. The interface may need to be updated to improve usability and make prediction quality information easier to understand.

13 Unlikely Changes

LC6: The fundamental purpose of DAIIM is unlikely to change. The system will continue to provide a marketplace for requesting AI inference and obtaining aggregated predictions from participating providers.

LC7: The system is unlikely to move away from its intended web-based form during the scope of this project. Users will continue to access the marketplace through a modern web browser with an internet connection.

LC8: The project is unlikely to introduce a physical hardware component. DAIIM will remain a software-based system that coordinates inference requests, provider predictions, and marketplace operations.

14 Appendices

14.1 Appendix A: Additional Materials

14.1.1 Weighted-Median Aggregation Algorithm

Only valid predictions revealed by providers with a non-zero reputation score at the end of the reveal phase are included. This reputation snapshot is taken before any reputation updates for that inference request. Each included prediction's weight is its provider's reputation score divided by the sum of the included providers' reputation scores. The weights are positive and sum to 1. FR31 requires at least the task's minimum number of included predictions before aggregation begins.

```
Algorithm WeightedMedian
Input:  Valid predictions p[1..n] from providers with non-zero reputation,
        and corresponding normalized weights w[1..n] where each w[i] > 0
        and sum(w[i]) = 1
Output: The weighted median

1.  Create a list L of pairs (p[i], w[i]) for i = 1 to n.
2.  Sort L in nondecreasing order by prediction value.
3.  Set cumulativeWeight <- 0.
4.  Set previousPrediction <- NONE.
5.  For each pair (value, weight) in L, in sorted order:
6.      previousWeight <- cumulativeWeight.
7.      cumulativeWeight <- cumulativeWeight + weight.
8.      If cumulativeWeight > 0.5:
9.          If previousWeight = 0.5:
10.             result <- (previousPrediction + value) / 2.
11.         Else:
12.             result <- value.
13.         Return result.
14.     Set previousPrediction <- value.
15. Return an aggregation error.
```

Read the predictions in increasing order, adding their weights as you go. The result is the prediction where the running total first goes above 0.5. If the running total was exactly 0.5 just before that prediction, use the average of that prediction and the preceding prediction instead.

14.1.2 Confidence-Based Reputation Update Algorithm

The reputation snapshot used by this algorithm is the score of each included provider at the end of the reveal phase, before any reputation updates for the request. The final aggregated

prediction F is the weighted median used as the center of the reputation-weighted variance σ^2 of the included predictions. If $\sigma^2 = 0$, every included prediction equals F; in this case each provider's reputation change is defined to be zero to avoid division by zero.

```
Algorithm ConfidenceBasedReputationUpdate
Input:  Predictions p[1..n] from providers with corresponding
        non-zero reputations r[1..n], and final aggregated prediction F
Output: Updated reputations for all included providers

1.  For each provider i, calculate d[i] <- abs(F - p[i]).
2.  Calculate σ^2 <- sum(r[i] * (p[i] - F)^2) / sum(r[i]).
3.  If σ^2 = 0, set delta[i] <- 0 for every provider and go to step 6.
4.  Calculate σ <- sqrt(σ^2).
5.  For each provider i, calculate delta[i] <- REPUTATION_ADJUSTMENT_FACTOR * (1 - d[i] /
6.  For each provider i, set r_new[i] <- min(100, max(0, r[i] + delta[i])).
```

The existing reputation range requirement in FR40 applies to the resulting reputation scores.

14.2 Appendix B: Development Plan

The development plan is not included in this SRS; see the separate Development Plan document.

14.3 Appendix C: Values of Auxiliary Constants

- REPUTATION_ADJUSTMENT_FACTOR: the multiplicative factor used to calculate reputation changes in the Confidence-Based Reputation Update Algorithm; 1.0.
- USABILITY_COMPLETION_RATE: the minimum fraction of usability test participants who must complete each task without assistance (NFR**??**). To be determined.
- MINIMUM_PROVIDER_STAKE: the minimum stake a provider must maintain for an inference task to remain registered for that task (FR51, FR32). To be determined.
- SLASH_FRACTION: the fraction of a provider's stake slashed for an invalid prediction or for not submitting a prediction (FR48, FR49, FR50). To be determined.
- REPEAT_OFFENSE_COUNT: the number of slashing offences that causes a provider to be unregistered (FR52). To be determined.
- COMMIT_PERIOD: the protocol-wide maximum time for accepted providers to submit commitments after the acceptance period ends (FR15). To be determined.
- REVEAL_PERIOD: the protocol-wide maximum time for committed providers to reveal after the commit phase ends (FR16). To be determined.

- PENDING_TRANSFER_PERIOD: the time after the end of a request's reveal phase during which unresolved reward transfers can be retried before the held reward is burned (FR45). To be determined.
- REPEAT_OFFENSE_WINDOW: the number of most recent accepted inference requests over which slashing offences are counted (FR52). To be determined.

14.4 Appendix D: Generative AI Disclosure

This section provides an overview of how generative AI tools were used in the making of this document.

AI Tools Used

- Claude Opus 5.5 (Anthropic), used through Claude Code.

How and Where Used

Claude Opus 5.5 was used throughout this document to help ideate and draft its content. The team provided the tool with the Problem Statement and Goals and the Development Plan, and used it to brainstorm the stakeholders, stakeholder requirements, functional requirements, and non-functional requirements that the document should address, and to produce first drafts of the text. In the Functional Requirements section, for example, it helped split broad descriptions of the marketplace's behaviour into atomic, testable requirements, grouped by the phase of an inference request they belong to, and it suggested symbolic constants, such as SLASH_FRACTION, for values that are not yet decided. No skills or custom agents were created to review this document. It was also used to draft the text of this appendix.

The tool was used as an aid, not as a replacement for the team's own work. Team members were involved in every sentence of the document: each AI-assisted draft was read, discussed, and edited by the team, and ideas that did not fit the project were discarded or rewritten.

Verification of AI-Generated Content

Every sentence in this document was reviewed by at least one team member before it was included. AI-assisted content was checked against the Problem Statement and Goals and the Development Plan to confirm that it describes behaviour the team intends DAIIM to have, that it does not introduce requirements or design decisions the team has not agreed to, and that it is consistent with the technologies the Development Plan lists. Cross-references between requirements were checked by hand, and each requirement was reviewed to confirm that it is atomic, testable, and does not prescribe a design. Values that are still undecided are left as symbolic constants in Appendix C and will be set once the proof of concept has been evaluated. The team remains responsible for the accuracy, completeness, and consistency of

this document, regardless of whether material was first drafted by a team member or with the help of an AI tool.

14.5 Appendix E: Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

1. What went well while writing this deliverable?

   Nathan: For the SRS and Hazard Analysis documents, we were able to learn a lot about our project by identifying potential hazards and discussing how we could mitigate them. For example, one of our main concerns was determining how the system would handle collusion attacks, which we discussed further during a meeting with our supervising professor. Through ongoing discussions among team members and with our professor, we were able to identify potential hazards and consider appropriate mitigation strategies We took a similar approach when developing the SRS, first identifying our functional requirements with guidance from our professor and ensuring that the different aspects of our project's scope were adequately represented. Overall, maintaining consistent communication within our team and with our professor allowed us to make incremental progress, receive regular feedback, and address uncertainties as they arose. This helped us ensure that our documents accurately reflected our project's intended scope and that we were progressing in the right direction.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   Nathan: One pain point our team encountered during this deliverable was determining the appropriate level of detail to include in certain sections. For example, when writing the critical assumptions in the Hazard Analysis, we were unsure whether explicitly stating that the system required at least one non-malicious inference provider would be too restrictive. Making this a formal assumption could imply that the system depended on this condition to operate correctly, potentially requiring additional assumptions or safety measures to address the risks associated with malicious providers. To address this, we decided to treat malicious provider behaviour as a potential hazard within the Failure Mode and Effects Analysis rather than making the presence of a non-malicious provider a concrete assumption. This allowed us to acknowledge and analyze the risk without assuming that malicious behaviour could be ruled out.

   Another pain point was leaving our team review call until the final two days before the deadline. The call lasted over three hours, making it difficult to coordinate everyone's availability and review the document efficiently. For future deliverables, we plan to create pull requests and have our sections ready for review three to four days before the deadline. This will allow team members to review each other's work at times that fit their schedules, provide feedback incrementally, and avoid relying on a single, lengthy meeting close to the deadline.

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

   Aidan: Most of our stakeholder requirements were inspired by speaking to our supervisor, who acts as our client, and by discussions with peers who represent the two kinds of users of DAIIM. Of the 15 stakeholder requirements in the Stakeholder Requirements section, roughly 12 came directly out of these conversations:

   - **Task requesters (SR1 to SR8):** Conversations about who would actually use a decentralized inference marketplace, such as emergency management agencies forecasting river levels, led us to focus on consensus and reliability. This inspired the requirements that results combine predictions from multiple independent providers (SR3), remain available when some providers are offline (SR4), come with information about their quality (SR5), and are not controlled by a single organization (SR6). Peers without a blockchain background pointed out that requesters should not need to understand decentralized systems to use DAIIM (SR8), and that they should be able to see and verify the payments they authorize (SR7).
   - **Inference providers (SR9 to SR12):** Discussions with our supervisor about why anyone would contribute their models to the network shaped all four provider requirements, especially that providers must be able to earn a profit (SR10) and that honest, accurate predictions must be more profitable than dishonest ones (SR11). These later drove most of the staking, reputation, reward, and penalty functional requirements.

   The three secondary stakeholder requirements on data privacy, protection, and retention (SR13 to SR15) were inspired less by direct conversations and more by our own research into regulatory expectations and by the hazards identified in the Hazard Analysis. Similarly, most of the detailed functional requirements, such as the exact rules for acceptance periods, reputation updates, and slashing, were derived by the team from the stakeholder requirements rather than requested directly. In this sense, client and peer conversations set the goals of the system, while the team translated those goals into specific, testable behaviour.

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

   Aidan: I will be focusing on the architecture and endpoints of DAIIM's backend server, and three of the software engineering courses I have taken will help with this work.

   - **SFWRENG 2AA4 (Software Design I — Introduction to Software Development):** This course taught me how to break a system into modules with clear responsibilities and well-defined interfaces, and how to apply design principles such as information hiding, low coupling, and high cohesion. These ideas will

guide how I separate the backend's concerns, such as handling inference requests, communicating with providers, and submitting transactions to the on-chain programs, so that each part can be developed and tested independently.

- **SFWRENG 3A04 (Software Design III — Large System Design):** This course covered architectural styles, design patterns, and how the components of a large system interact. It will help me choose an architecture for the backend that fits DAIIM's mix of off-chain and on-chain components, and design endpoints that give the frontend and inference providers a clear, consistent interface to the system. It also prepared me to document these decisions in the design documents later in the project.
- **SFWRENG 2C03 (Data Structures and Algorithms):** This course taught me how to choose data structures and analyze the efficiency of algorithms. This will help me design backend logic that stays efficient as the number of tasks, requests, and providers grows, such as tracking the phase of each active inference request, matching requests to registered providers, and collecting predictions before aggregation.

Nathan: A course I have taken in the past that will help our team succeed with this capstone project is 3RA3, which focused on software requirements engineering. In this course, we learned about goal modelling and how to distinguish high-level stakeholder goals from concrete software requirements. This knowledge has been useful throughout the capstone project because it has allowed me to better understand what makes a good goal and how to refine it into requirements that describe what our system should accomplish. For example, one of DAIIM's main goals is to provide reliable aggregated inference results. Rather than treating this broad goal as a requirement on its own, we needed to identify the specific system responsibilities that would help achieve it, such as aggregating provider predictions and maintaining provider reputations. Additionally, the knowledge I gained about identifying stakeholders and their goals has helped me consider the different needs of task requesters and inference providers, particularly when designing a marketplace that must balance prediction accuracy with fair provider compensation. Overall, the concepts I learned in 3RA3 have helped me contribute to defining our project's scope and developing requirements that align with its intended goals

Robert: I am currently taking COMPSCI 4CR3 (Applied Cryptography), and the course will help me understand the cryptographic foundations of the project's blockchain and cryptocurrency components. Later in the course, we will discuss blockchain technology, which will help me better understand how DAIIM's blockchain components are implemented. Specifically, the security aspects of private-key cryptography and digital signatures are important to the project. A private key is used to sign transactions, allowing users to authorize actions with their wallets. Understanding these mechanisms will help me reason about how authorization and escrow work in DAIIM, and why protecting private keys is essential. Furthermore, having a solid understanding of key

signing will be useful in creating a user interface that is able to abstract away a lot of this complexity while still giving users enough information to understand the actions they are taking.

Keira: Of the courses I have taken throughout my undergraduate degree, I believe the following three will help my team be successful in this capstone project:

- **(Software Design III – Large System Design):** In this course, I learned crucial information about sustainable architectures, methodology of making software architecture decisions, and gained experience designing and developing a complex full-stack program. This included identifying requirements, creating a development plan, and implementing the system. I can use the knowledge and experience gained from this course to navigate the design process, making design decisions, and the implementation of the capstone project.
- **4C03 (Computer Networks and Security):** In this course, I learned about different network protocols, as well as encryption and information security. This knowledge is crucial in addressing some of the hazards we identified and meeting some functional and nonfunctional requirements related to data handling and network transactions.
- **4HC3 (Human Computer Interfaces):** I am currently enrolled in this course and I believe the content related to usability requirement development and designing a user interface that meets the needs of the target user will be helpful, as a huge part of our capstone project is related to the web client through which requesters will interact with the system. Some major nonfunctional requirements which will verify if the project is a success with the target requester user group involve making the technical aspects of the system, such as the blockchain and Web3 wallet infrastructure, accessible and learnable to the average user with limited knowledge in these areas.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

   Much of the knowledge and skills our team will need to collectively acquire to successfully complete this capstone project are related to domain specific knowledge and specific expected technology. This project is highly technical and we will be using tools that we do not have previous experience of implementing.

   Having worked with Dr. Hassan before, Robert was the only member with experience using Solana Devnet and Solana Mainnet to test and implement blockchain transactions and programs. Aidan and Keira had no previous knowledge of or experience with blockchains or smart contracts, and had overcome this learning curve through research.

Robert was able to share resources that he had cited previously in his research work, which helped support their learning at the beginning of the project.

The token economics is another area with which all members do not have or have little experience. As most members do not have experience with Solana, blockchain, or wallets, it naturally follows that experience using SOL tokens on blockchain is something that will have to be acquired for the success of the project.

None of our four members have implemented an embedded Web3 wallet in our previous projects. Our supervisor, Dr. Hassan, made us aware of the availability of the tool for embedding the wallet in a way that is more familiar to the average user with no experience with Web3 wallets. This will help support our usability non-functional requirements for the users.

Additionally, a major part of the project is determining a reliable aggregation algorithm for regression. This requires foundational knowledge in theoretical mathematics and precision errors. Malicious providers and collusion attacks also pose a major risk to the system. Understanding methods to combat these risks is necessary for the success of the system and completion of the project.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   The team will use a combination of independent study and practical experimentation to build the knowledge needed for DAIIM.

   - **Blockchain, smart contracts, and wallets:** One approach is to study the official Solana, Anchor, and wallet documentation, along with cryptography resources on keys and transaction signatures. A second approach is to build and test small programs and wallet flows on Solana Devnet, including transactions that lock and release test funds from escrow. The team can follow this Solana escrow tutorial as a practical starting point. Robert will draw on his prior Solana experience and his Applied Cryptography coursework to guide the team's understanding of transactions, signatures, and escrow security. The other team members will use a combination of reading documentation and following these practical tutorials to get up to speed on this topic.
   - **Token economics and incentives:** The team can learn through researching existing web3 and decentralized application that use a similar token curated registry mechanism and by discussion with the supervisor. Our supervisor has direct experience building a decentralized learning platform for educators and learners, and several connections to other developers who worked on this project. Robert and Kiera will focus on traditional research techniques. All four members will

participate in discussions with the supervisor and other developers to ask questions, learn from their experience, and understand practical considerations for designing the system's token economics.

- **Regression aggregation** One approach is to study statistical measures and aggregation methods, including how outliers and numerical precision affect results. A second is to implement experiments using synthetic predictions and known ground truth, varying the proportions of accurate, inaccurate, and colluding providers. All members will contribute to the proof of concept (POC), with Robert leading from experience with his previous project. Aidan will conduct additional research on regression aggregation methods to inform the design and evaluation of the simulations.
- **Embedded wallets:** The frontend developers can first study the wallet integration documentation and example applications, then build a small prototype and test the connection, signing, and transaction status flows. Aidan and Nathan will focus on learning and prototyping the integration, while the rest of the team will review the flow and give feedback from the perspective of users who are less familiar with Web3.